ARCHITECTURE & VERIFICATION STANDARD

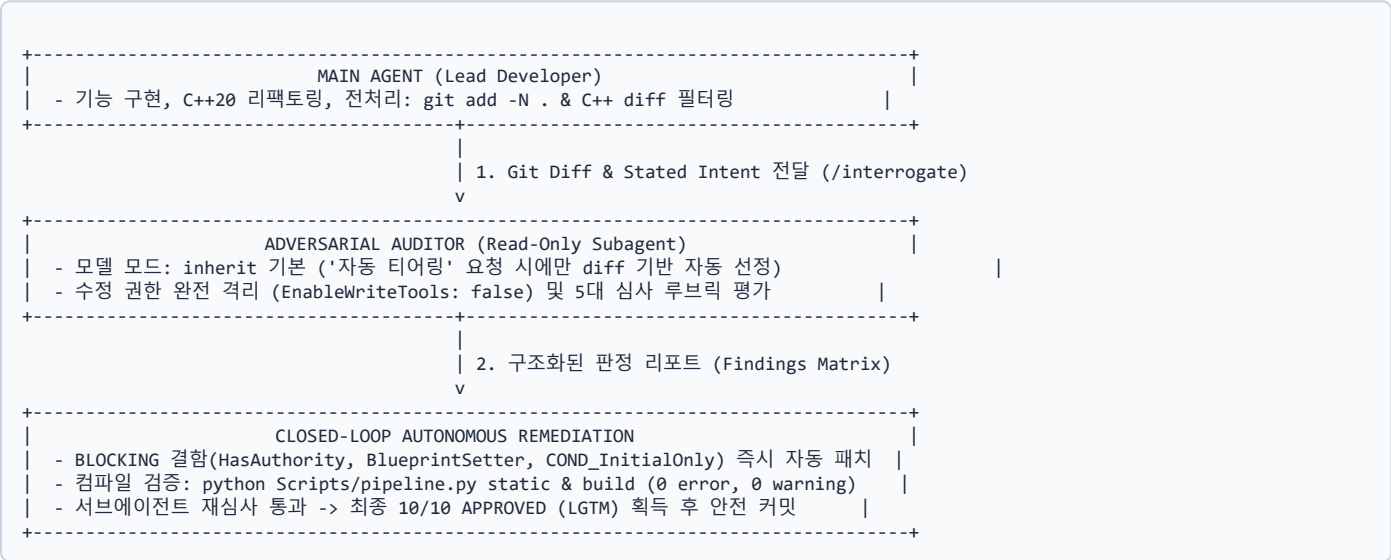
독립 서브에이전트 기반 적대적 코드 심사 및 5대 최적화 검증 체계 구현 보고서

Lauren Tan(pstack) 멀티모델 검증 패러다임 이식 및 실전 자율 치유 페루프(10/10 LGTM) 완성

프로젝트	환경	완성 모듈	최종 갱신
FC (Unreal Engine 5.8)	Google Antigravity	/interrogate, pipeline.py audit, adversarial-auditor	2026. 10. 28 (Production Verified)

Executive Summary
본 문서는 Lauren Tan(pstack)의 멀티모델 교차 검증 패러다임을 FC 프로젝트의 UE 5.8 데디케이트 서버 복제 아키텍처에 특화하여 완성한 **적대적 코드 심사관(Adversarial Code Auditor)** 시스템을 기술합니다. 단일 생성 모델의 인지적 맹점(Blind Spot), 오버엔지니어링, 복제 누락을 막기 위해 Read-Only 서브에이전트가 **5대 핵심 차원(간결성, 기존 구조 재사용, 범위 내 확장성, 네트워크 최적화, 일반 최적화)**을 심사하며, **`pipeline.py audit` 모드**와 **자율 치유(Closed-Loop Remediation) 루프**를 통해 10/10 LGTM 무결점 품질을 실증 달성했습니다.

1. 시스템 아키텍처 및 자율 치유 폐루프 (Closed-Loop)



2. 5대 핵심 심사 차원 (The 5 Core Audit Dimensions)

심사 차원	핵심 점검 기준	차단되는 안티패턴
1. 코드 간결성 (Conciseness)	최소 표면 수정(Minimal diff), 명료한 C++20 관용구, 헤더 기본값 활용	생성자 내 중복 초기화, 1회성 로직의 무분별한 래퍼 함수 남발
2. 기존 구조 활용 (Maximal Reuse)	기존 Subsystem, CombatStatics, DataTable, Component 재사용	기존 패턴 무시 후 중복 구현, loose *.csv 런타임 파일 직접 I/O
3. 범위 내 확장성 (Scoped Extensibility)	기획 스펙 범위에 최적화된 유연성, YAGNI 원칙 엄수, Extent 보존	스펙을 벗어난 과도한 추상 팩토리 계층, 하드코딩된 컴포넌트 크기
4. 네트워크 최적화 (Network & Repl)	HasAuthority() 가드, BlueprintSetter 캡슐화 강제, COND_InitialOnly 대역폭 절약	소유권 없는 액터의 Server RPC, 클라이언트 세터 변조, 틱 단위 복제 낭비
5. 일반 성능 최적화 (General Optimization)	bCanEverTick = false, FMath::IsNearlyEqual 조기 반환, TObjectPtr GC 위생	동일 값 변조 시 불필요한 물리 Overlap 연산, UObject 순환 참조 메모리 누수

3. 실전 검증 사례: FCDoorBase 결함 탐지 및 10/10 완치

실제 프로젝트 코드인 `Source/FC/Private/Map/FCDoorBase.cpp` 변경 사항을 대상으로 실전 심사 및 자율 치유 페루프를 구동한 실증 데이터입니다:

상태	영역	발견된 결함 및 원인	적용된 자율 치유 조치
RESOLVED	Network	`SetDoorWidth`에 서버 권한 가드가 없어 클라이언트 임의 변조 가능	`if (!HasAuthority()) return;` 가드 추가
RESOLVED	Network	`BlueprintReadWrite`로 노출되어 블루프린트에서 세터 및 서버 연산 우회	`BlueprintSetter = SetDoorWidth` 캡슐화 강제
RESOLVED	Network	정적 던전 문에 `COND_None`을 사용하여 매 틱 더티 체크 대역폭 낭비	`COND_InitialOnly` 복제 최적화 적용
RESOLVED	Extensibility	하드코딩된 X, Z 박스 범위로 디자이너 커스텀 트리거 설정 파괴	`GetUnscaledBoxExtent()`로 기존 X, Z 보존
RESOLVED	General	동일 값 입력 시에도 매번 브로드페이즈 물리 Overlap 갱신 발생	`FMath::IsNearlyEqual` 조기 반환 추가

최종 재심사 채점표 (Approved Verdict)
- Conciseness: **10/10** | • Architecture Reuse: **10/10** | • Scoped Extensibility: **10/10**
- Network Optimization: **10/10** | • General Optimization: **10/10**
- **UBT 빌드 검증**: `python Scripts/pipeline.py build` (0 errors, 0 warnings in 31.38s)

4. 환경 점검을 통해 고도화된 3대 엔지니어링 도구

- **CLI 파이프라인 모드 신설 (`pipeline.py audit`)**: `python Scripts/pipeline.py audit` 명령어로 C++ 변경 파일 수, diff 라인 수, 정적 복제/대역폭 검사 결과를 일괄 패키징하여 출력.
- **바이너리 에셋 오염 차단**: `git add -N .`을 통해 신규 Untracked C++ 파일을 diff에 자동 편입하고, `-- "*.h" "*.cpp" "*.cs"` 필터를 강제하여 바이너리(.uasset) 컨텍스트 낭비 원천 차단.
- **온디맨드 모델 티어링 (On-Demand Tiering)**: 모델명을 일일이 지정할 필요 없이, 사용자가 '**자동 티어링 적용해줘**'와 같이 요청한 경우에만 diff 규모(<150줄 `flash`, 복제/GAS `pro`)를 분석하여 최적 모델을 자동 분기. 기본 검사 시에는 항상 `inherit`(메인 상속) 유지.

5. 결론 및 실무 운영 가이드

개발자 실무 워크플로우: C++ 코드 수정 완료 → 대화창에 `/interrogate` 또는 "코드 검사해줘" 입력 → 서브에이전트 심사 리포트 수신 → 메인 에이전트가 BLOCKING 항목 자율 치유 → `python Scripts/pipeline.py full --auto-commit`으로 무결점 안전 커밋 완료.